

CIC0198 - TÉCNICAS DE PROGRAMAÇÃO 2

Arquiteturas de Software

Daniel Porto – daniel.porto@unb.br
Departamento de Ciência da Computação – CIC
Universidade de Brasília – UnB

Material adaptado do Prof. Marco Tulio Valente



UnB | IE | CIC

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

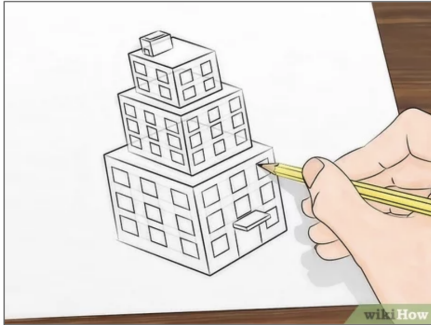
Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

- ▶ Projeto em mais alto nível
- ▶ Foco **não** são mais unidades pequenas (ex.: classes)
- ▶ Mas sim unidades maiores e mais relevantes
 - Pacotes, módulos, subsistemas, camadas, serviços, ...

SOFTWARE ARCHITECTURE



Software Design



Software Architecture

PADRÕES ARQUITETURAIS

- ▶ Modelos pré-definidos para arquiteturas de software
- ▶ Vamos estudar:
 - Camadas (duas e três camadas)
 - Model-View-Controller (MVC)
 - Microserviços
 - Orientada a Mensagens
 - Publish/Subscribe

UMA VEZ TEVE UMA TRETA...



DEBATE LINUS-TANENBAUM (1992)



Criador do sistema
operacional Linux



Autor de livros e do
sistema operacional
Minix

INÍCIO DO DEBATE: MENSAGEM DO TANENBAUM (1992)

From: ast@cs.vu.nl (Andy Tanenbaum)
Newsgroups: comp.os.minix
Subject: **LINUX is obsolete**
Date: 29 Jan 92 12:12:50 GMT

I was in the U.S. for a couple of weeks, so I
LINUX (not that I would have said much had I
it is worth, I have a couple of comments now.

ARGUMENTO DO TANENBAUM

- ▶ Linux possui uma **arquitetura monolítica**
 - › Sistema operacional é um único arquivo
 - › Gerência de processos, memória, arquivos, etc
- ▶ O melhor seria uma **arquitetura microkernel**
 - › Kernel só possui serviços essenciais
 - › Demais serviços rodam como processos independentes

RESPOSTA DO LINUX

```
From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
Subject: Re: LINUX is obsolete
Date: 29 Jan 92 23:14:26 GMT
Organization: University of Helsinki
```

```
Well, with a subject like this, I'm afraid I'll have to reply.
```

ARGUMENTO DO LINUS

Em teoria, arquitetura microkernel é mais interessante

Mas existem outros critérios que devem ser considerados

Um deles é que o Linux já era uma realidade e não apenas uma promessa

NOVA MENSAGEM DO TANENBAUM

“Eu continuo com minha opinião...”

“Agradeça por não ser meu aluno. Se fosse, você não iria tirar uma nota alta com essa arquitetura.”

COMENTÁRIO DO KEN THOMPSON (UNIX)

“É mais fácil implementar um SO com um kernel monolítico.”

“Mas é também mais fácil que ele se transforme em uma bagunça à medida que o kernel é modificado.”

KEN THOMPSON PREVIU O FUTURO

17 anos depois (2009) veja a
declaração de Torvalds em uma
conferência de Linux



DECLARAÇÃO DE TORVALDS (2009)

“Não somos mais o kernel simples, pequeno e hiper-eficiente que imaginei há 15 anos.”

“Em vez disso, o kernel está grande e inchado. Quando adicionamos novas funcionalidades, o cenário piora.”

Is Linux kernel getting bloated ? Linus Torvalds says Yes!

September 24, 2009 Posted by Ravi

MORAL DA HISTÓRIA:

Os custos de uma decisão arquitetural podem levar anos para aparecer...

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

CONCEITO DE CAMADAS

Sistema é organizado em camadas, de forma hierárquica

Camada n somente pode usar serviços da camada $n-1$

Muito usada em redes computadores e sist. distribuídos

OSI model		
Layer	Name	Example protocols
7	Application Layer	HTTP, FTP, DNS, SNMP, Telnet
6	Presentation Layer	SSL, TLS
5	Session Layer	NetBIOS, PPTP
4	Transport Layer	TCP, UDP
3	Network Layer	IP, ARP, ICMP, IPsec
2	Data Link Layer	PPP, ATM, Ethernet
1	Physical Layer	Ethernet, USB, Bluetooth, IEEE802.11

- ▶ Dividir para conquistar:
 - Quebra complexidade do sistema
 - Facilita entendimento
 - Facilita troca de camadas (ex: TCP, UDP)
 - Facilita reuso de camadas (ex: várias apps usam TCP)

VARIAÇÕES PARA SISTEMAS DE INFORMAÇÕES

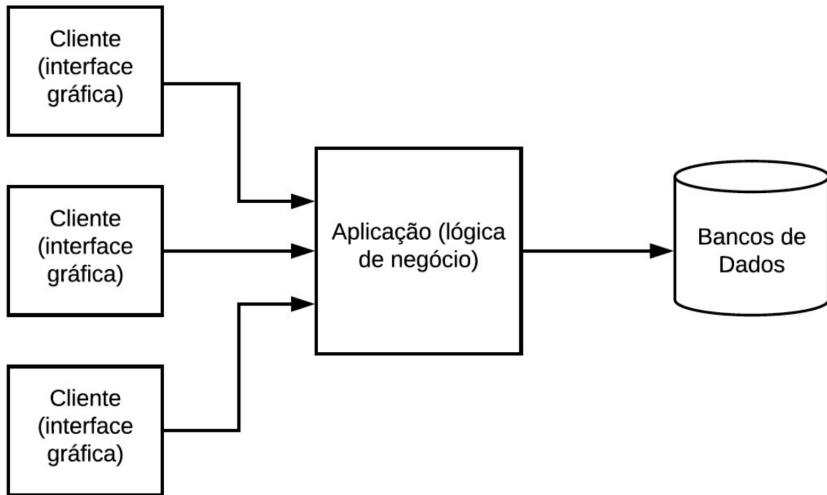
- ▶ Três camadas
- ▶ Duas camadas

ARQUITETURA EM TRÊS CAMADAS

- ▶ Comum em processos de *downsizing* de aplicações corporativas nas décadas de 80 e 90
- ▶ Downsizing: migração de computadores de mainframes para servidores, rodando Unix



ARQUITETURA EM TRÊS CAMADAS



ARQUITETURA EM DUAS CAMADAS

- ▶ Mais simples:
 - Camada 1 (cliente): interface + lógica
 - Camada 2 (servidor de BD): bancos de dados
- ▶ Desvantagem: todo o processamento é feito no cliente

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

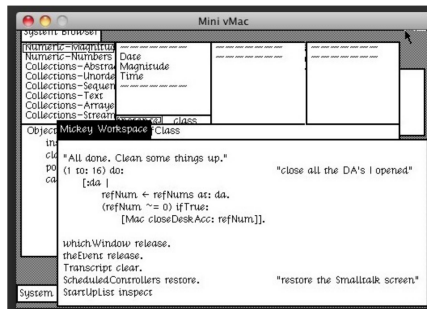
Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

ARQUITETURA MVC

- ▶ Surgiu na década de 80, com a linguagem Smalltalk
- ▶ Proposta para implementar interfaces gráficas (GUIs)



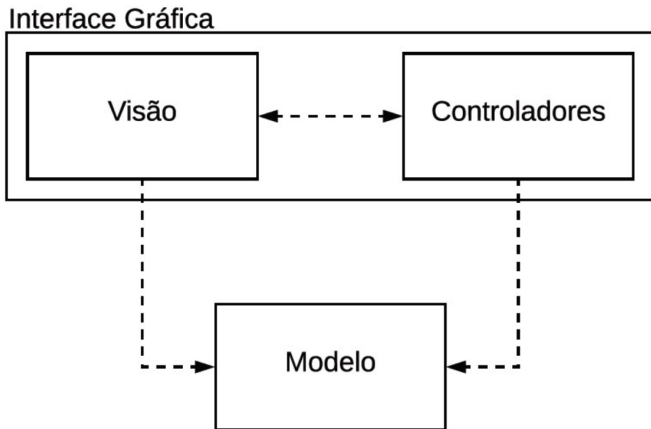
Visão: classes para implementação de GUIs, como janelas, botões, menus, barras de rolagem, etc

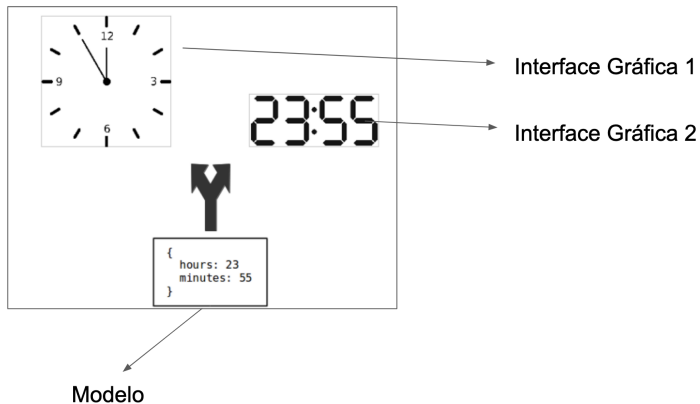
Controle: classes que tratam eventos produzidos por dispositivos de entrada, como mouse e teclado

Modelo: classes com lógica e dados da aplicação

MVC

MVC = (Visão + Controladores) + Modelo = Interface Gráfica + Modelo

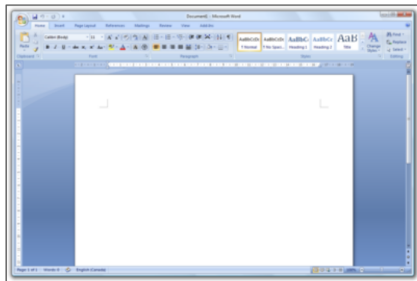




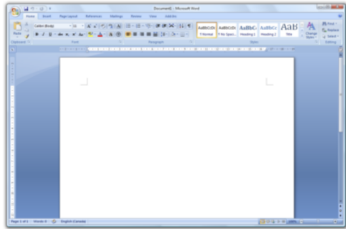
MVC - IMPORTANTE!

MVC não foi pensado para aplicações distribuídas; mas para aplicações desktop monolíticas

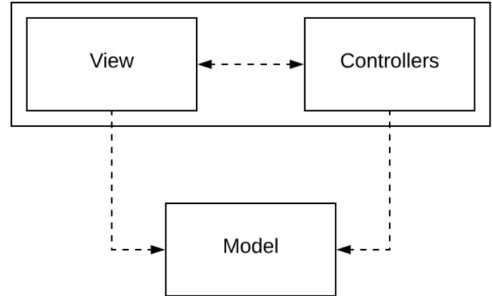
Exemplo: Microsoft Word, Calculadora, etc



MVC TRADICIONAL (SMALLTALK)



Graphical Interface



MVC NOS DIAS DE HOJE

- ▶ MVC Web
- ▶ Single Page Applications

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

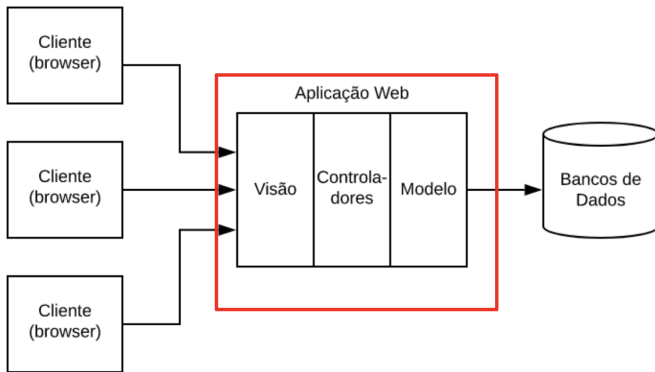
Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

Adaptação de MVC para Web, ou seja, sistemas distribuídos
Usando Ruby on Rails, Django, Spring, PHP Laravel, etc

MVC WEB



Visão: Páginas HTML, CSS, JavaScript (o que o usuário vai ver)

Controladores: Recebem dados de entrada e fornecem informações para páginas de saída

Modelo: Lógica da aplicação (regras de negócio) e fazem a interface com o banco de dados

CONTROLADOR - EXEMPLO 1

```
1 public class ControladorPesquisaLivros {  
2     ...  
3     public void start() {  
4         ...  
5         get("/", (req, res) -> {  
6             res.redirect("index.html");  
7             return null;  
8         });  
9         ...  
10    }  
11 }
```

Biblioteca MVC

Pesquisa de Livros

Informe o nome do autor

Pesquisar

Nomes válidos: valente, fowler e gof

CONTROLADOR - EXEMPLO 2

```
1 public class ControladorPesquisaLivros {
2     ServicoPesquisaLivros pesq;
3     PaginaDadosLivro pagina;
4     ...
5     public void start() {
6         ...
7         get("/pesquisa", (req, res) -> {
8             String autor = req.queryParams("autor");
9             Livro livro = pesq.pesquisaPorAutor(autor); ◀◀
10            return pagina.exibeLivro(livro.getTitulo(),
11                                    livro.getAutor(), livro.getISBN());
12        });
13        ...
14    }
15 }
```

MODELO - EXEMPLO

```
1 public class ServicoPesquisaLivros {
2     public Livro pesquisaPorAutor(String autor) {
3         try(Connection con = DriverManager.getConnection(...)) {
4             String query = "select * from livros where autor = ?";
5             PreparedStatement stmt = con.prepareStatement(query);
6             stmt.setString(1, autor);
7             ResultSet rs = stmt.executeQuery();
8             String isbn = rs.getString("isbn");
9             String titulo = rs.getString("titulo");
10            return new Livro(isbn, autor, titulo); ◀◀
11        } catch (SQLException e) {
12            System.out.println(e.getMessage());
13            return null;
14        }
15    }
16 }
```

CONTROLADOR - EXEMPLO 3

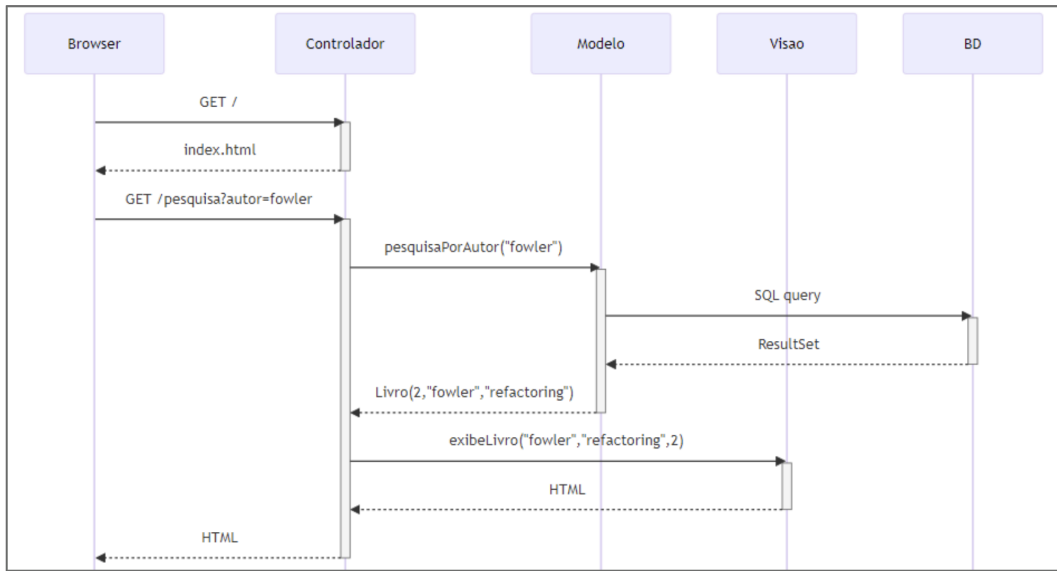
```
1 public class ControladorPesquisaLivros {
2     ServicoPesquisaLivros pesq;
3     PaginaDadosLivro pagina;
4     ...
5     public void start() {
6         ...
7         get("/pesquisa", (req, res) -> {
8             String autor = req.queryParams("autor");
9             Livro livro = pesq.pesquisaPorAutor(autor);
10            return pagina.exibeLivro(livro.getTitulo(), ◀◀
11                                     livro.getAutor(),
12                                     livro.getISBN());
13        });
14        ...
15    }
16 }
```

VISÃO - EXEMPLO

```
1 public class PaginaDadosLivro {
2     public String exibeLivro(String titulo, String autor, String isbn) {
3         String res = "<h4> Dados do Livro Pesquisado </h4>";
4         res += "<ul>";
5         res += "<li> Título: " + titulo + " </li>";
6         res += "<li> Autor: " + autor + " </li>";
7         res += "<li> ISBN: " + isbn + " </li>";
8         res += "</ul>";
9         return res;
10    }
11 }
```

Dados do Livro Pesquisado

- Título: Refactoring
- Autor: fowler
- ISBN: 2



OBSERVAÇÃO

MVC => Aplicação Monolítica (Monolito)

Inclusive, esse monolito possui frontend e backend integrados

MVC FRAMEWORKS CONTINUAM SENDO RELEVANTES!

You're in good company.

Over the past two decades, Rails has taken countless companies to millions of users and billions in market valuations.

 Basecamp

 HEY

 GitHub

 shopify

 instacart

 dribbble

 gusto

 zendesk

 airbnb

 Square

 KICKSTARTER

 HEROKU

 coinbase

 SOUNDCLLOUD

 cookpad

 doximity

 INTERCOM

 Fleetio

 Lime

 FreeAgent

<https://rubyonrails.org> (Maio 2025)

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

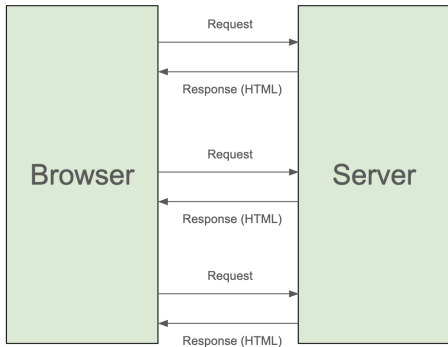
Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

APLICAÇÕES WEB TRADICIONAIS



Multiple Page Applications (MPAs)

Problema:
interface **menos responsiva**

SINGLE PAGE APPLICATIONS

Roda no browser; logo, mais independente do servidor

“Menos burra”: manipula sua interface e armazena dados

Acessa o servidor para buscar mais dados

Exemplo: GMail, Google Docs, Facebook, Figma, etc

Implementadas em JavaScript, usando React, Vue, Angular, etc

EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
    </button></p>
```

```
</div>
```

Interface (Web)
HTML

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Uma Simples SPA

Temperatura: 60

Incrementa

EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
    </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Modelo

Dados

Métodos

EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>

<div id="ui">
  Temperatura: {{ temperatura }}
  <p><button v-on:click="incTemperatura">Incrementa
  </button></p>
</div>

<script>
var model = new Vue({
  el: '#ui',
  data: {
    temperatura: 60
  },
  methods: {
    incTemperatura: function() {
      this.temperatura++;
    }
  }
})
</script>
```

EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>

<div id="ui">
  Temperatura: {{ temperatura }}
  <p><button v-on:click="incTemperatura">Incrementa
    </button></p>
</div>

<script>
var model = new Vue({
  el: '#ui',
  data: {
    temperatura: 60
  },
  methods: {
    incTemperatura: function() {
      this.temperatura++;
    }
  }
})
</script>
```


EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>

<div id="ui">
  Temperatura: {{ temperatura }}
  <p><button v-on:click="incTemperatura">Incrementa
    </button></p>
</div>

<script>
var model = new Vue({
  el: '#ui',
  data: {
    temperatura: 60
  },
  methods: {
    incTemperatura: function() {
      this.temperatura++;
    }
  }
})
</script>
```

RESUMO

MVC Tradicional

(Smalltalk): aplicações gráficas, pré-Web



MVC Web: adaptação de MVC para Web (fullstack)



SPA: adaptação de MVC para uso no front-end



React



Vue.js



ANGULAR

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

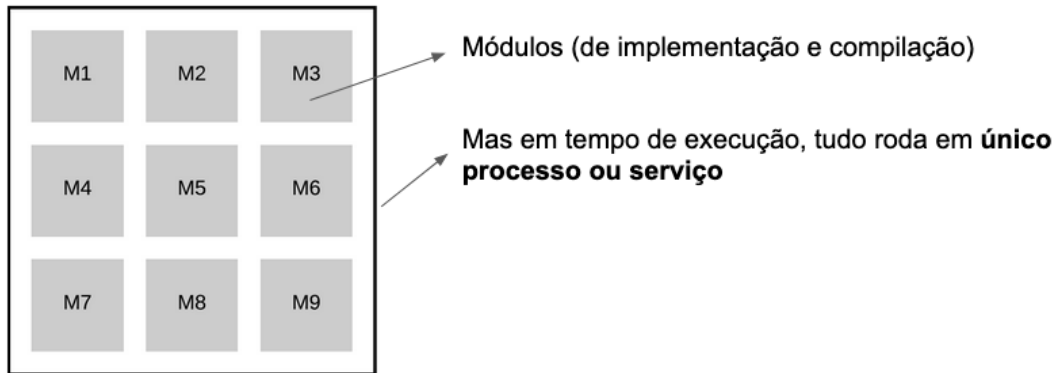
Exercícios

IMPORTANTE:

O que vamos estudar no restante do capítulo diz respeito a arquiteturas de backend

MONOLITOS

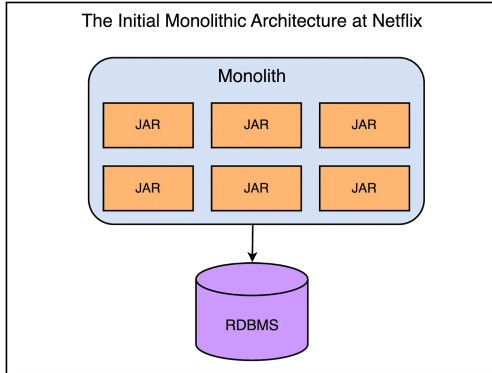
Em tempo de execução, sistema é um único processo (processo aqui é processo de sistema operacional)



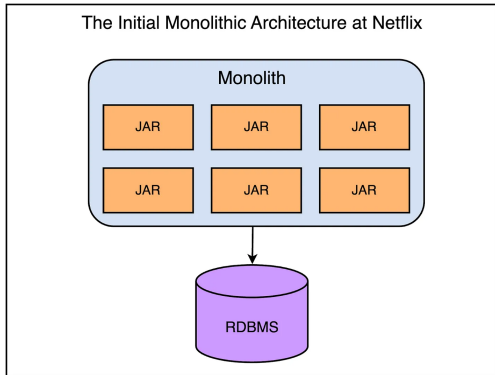
PROBLEMAS

Quais são os contras (possíveis problemas) de se projetar um software como um monolito?

PROBLEMA #1: SINGLE POINT OF FAILURE



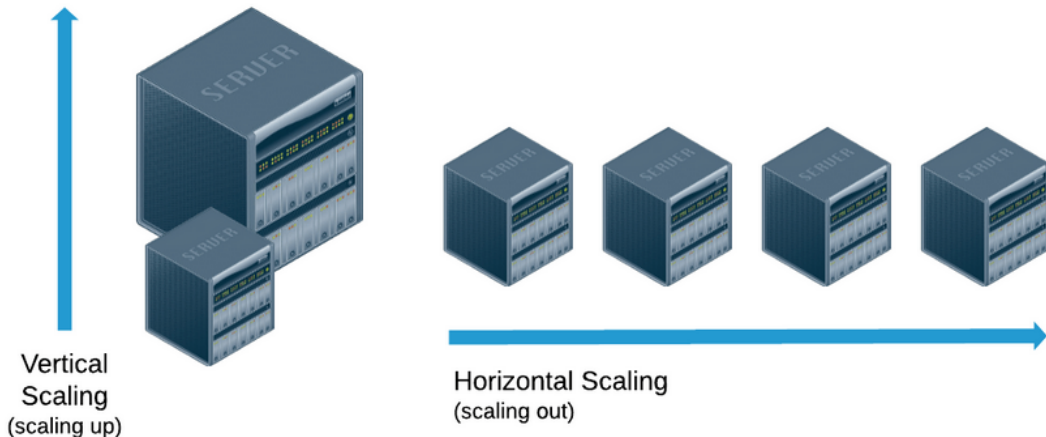
PROBLEMA #1: SINGLE POINT OF FAILURE



- ▶ 2008: grave falha no sistema (BD)
- ▶ Dias sem conseguir alugar DVDs
- ▶ Duas mudanças importantes:
 - AWS + microsserviços

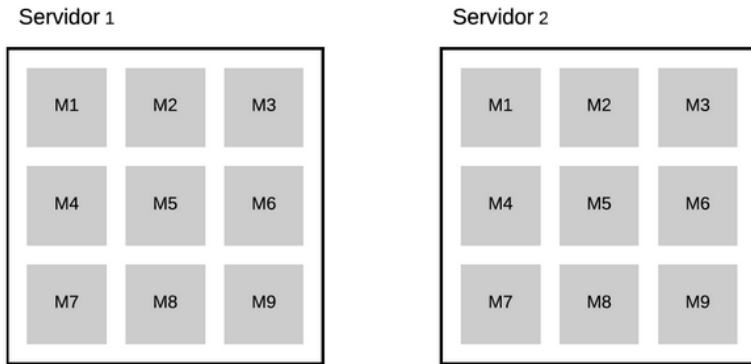
Fonte: <https://blog.bytebytego.com/p/a-brief-history-of-scaling-netflix>

ESCALABILIDADE: VERTICAL VS HORIZONTAL



PROBLEMA #2 COM MONOLITOS: ESCALABILIDADE

Escalabilidade horizontal requer escalar o monolito inteiro, mesmo quando o gargalo está em um único módulo



PROBLEMA #3: RELEASE É MAIS LENTO

- ▶ Times não tem poder para colocar módulos em produção
- ▶ Motivo: mudanças em um módulo podem impactar módulos que estão funcionando
- ▶ Acabam existindo:
 - Datas pré-definidas para release
 - Processo de homologação de releases

EXEMPLO

- ▶ Novo releases: sempre no dia 10
 - Último dia para mudanças: último dia do mês anterior
 - Testes e homologação: dia 1 a 10
- ▶ Se implementou uma feature no dia 01/09, mesmo que pequena, ela só entrará em produção no dia 10/11

OUTRA COISA: VAMOS ADICIONAR UMA PEQUENA FEATURE EM PRDUÇÃO...



OUTRA COISA: VAMOS ADICIONAR UMA PEQUENA FEATURE EM PRDUÇÃO...



Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

MICROSSERVIÇOS

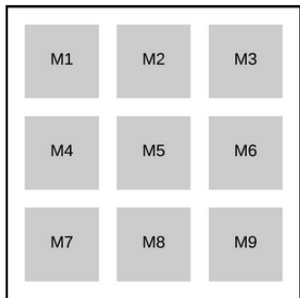
Módulos (ou conjuntos de módulos) viram processos em tempo de execução

Esses módulos são menores do que de um monolito

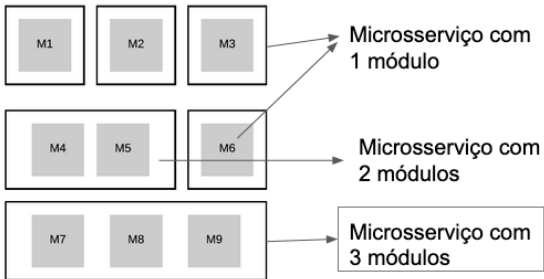
Daí o nome **microserviço**

COMPARAÇÃO: MONOLÍTICA VS MICROSERVIÇOS

Arquitetura Monolítica



Arquitetura baseada em Microserviços



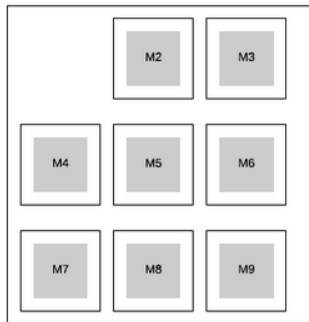
microserviço = processo (run-time, sistema operacional)

VANTAGEM #1: ESCALABILIDADE

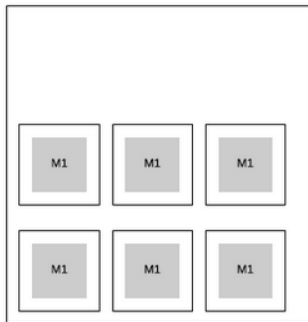
Pode-se escalar apenas o módulo com problema de desempenho

Só contém instâncias de M1, pois apenas ele é o gargalo de desempenho

Servidor 1



Servidor 2



Só contém instâncias de M1, pois apenas ele é o gargalo de desempenho



VANTAGEM #2: FLEXIBILIDADE PARA RELEASES

Times ganham autonomia para colocar microsserviços em produção

Processo = espaço de endereçamento próprio

Chances de interferências entre processos são menores

RELEASES DE MICROSERVIÇOS

Não existe mais uma homologação centralizada

Cada time homologa seus microsserviços

Se implementou uma feature no dia 01/09, ela pode entrar em produção imediatamente

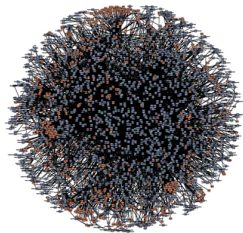
OUTRAS VANTAGENS

Tecnologias diferentes

Falhas parciais

QUEM USA MICROSERVIÇOS?

Grandes empresas...



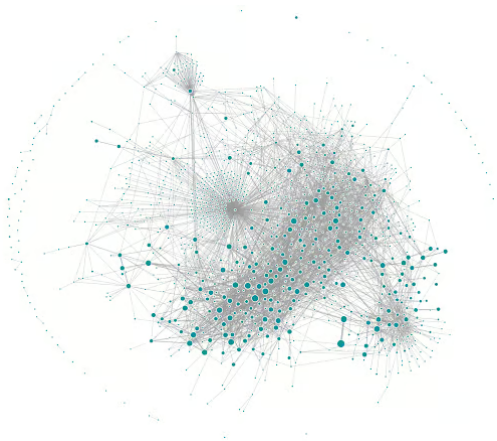
amazon.com



NETFLIX

Cada nodo é um microsserviço

UBER (~2018)



. IT Numbers

26
MM RPS

28K
Deploys
Day

120
Pull Requests
Day

35K+
Microservices

140 Clusters
230K Pods
17k Nodes

+16K
IT People

EXEMPLO DE UMA APLICAÇÃO DE ECOMMERCE BASEADA EM MICROSERVIÇOS

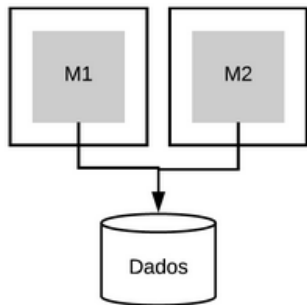
Team	Framework	Repository
Core	FastAPI	auth-api
Core	FastAPI	fraud-api
Core	FastAPI	group-watcher-api
Core	FastAPI	ledger-composer-api
Core	FastAPI	twilio-api
Core	FastAPI	user-api
Core	FastAPI	whatsapp-instance-api
Core	FastAPI	whatsapp-reader-api
Core	FastAPI	whatsapp-writer-api

EXEMPLO DE UMA APLICAÇÃO DE ECOMMERCE BASEADA EM MICROSERVIÇOS

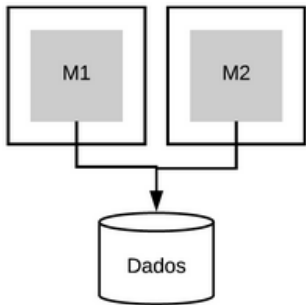
Fulfillment	FastAPI	community-api
Fulfillment	FastAPI	hub-api
Fulfillment	FastAPI	invoice-api
Fulfillment	FastAPI	package-api
Fulfillment	FastAPI	refund-api
Fulfillment	FastAPI	sorting-api
Fulfillment	FastAPI	supplier-exports-api
Fulfillment	FastAPI	supplier-payment-api
Fulfillment	FastAPI	support-ticketing-api
Fulfillment	FastAPI	tms-service-api
Fulfillment	FastAPI	wms-service-api

Consumer	FastAPI	coupon-api
Consumer	FastAPI	emporium-composer-api
Consumer	FastAPI	emporium-promo-api
Consumer	FastAPI	events-tagging-api
Consumer	FastAPI	google-tag-manager-api
Consumer	FastAPI	mgm-api
Consumer	FastAPI	orders-api
Consumer	FastAPI	product-recommendation-api
Consumer	FastAPI	product-review-api
Consumer	FastAPI	product-sample-api
Consumer	FastAPI	search-params-api
Consumer	FastAPI	user-lists-api
Consumer	FastAPI	user-notifications-api

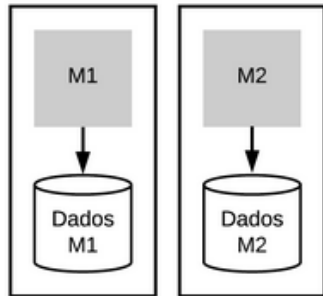
GERENCIAMENTO DE DADOS COM MICROSERVIÇOS



GERENCIAMENTO DE DADOS COM MICROSERVIÇOS



❌ Arquitetura que **não** é recomendada. Motivo: aumenta acoplamento entre M1 e M2



QUANDO NÃO USAR MICROSERVIÇOS?



QUANDO NÃO USAR MICROSERVIÇOS?

- ▶ Arquitetura com microserviços é mais complexa
 - Sistema distribuído (monitorar centenas de processos)
 - Latência (comunicação via rede)
 - Transações distribuídas

RECOMENDAÇÃO: COMEÇAR COM MONOLITOS

- ▶ E migrar para microsserviços se:
 - Monolito apresentar problemas de desempenho
 - Monolito estiver atrasando as releases
- ▶ Migração pode ser gradativa...

MAS EXISTEM EXCEÇÕES

Microserviços no Nubank, uma visão geral

O que aprendemos ao desenvolver um sistema complexo nos primeiros seis anos de uma fintech



12 Aug, 19

Software Engineering

by Rafael Ferreira – Software Engineer

“começamos com microserviços desde o primeiro dia, desafiando o conselho padrão de começar com um monólito.”

<https://building.nubank.com/pt-br/microservicos-no-nubank-uma-visao-geral/>



UnB

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

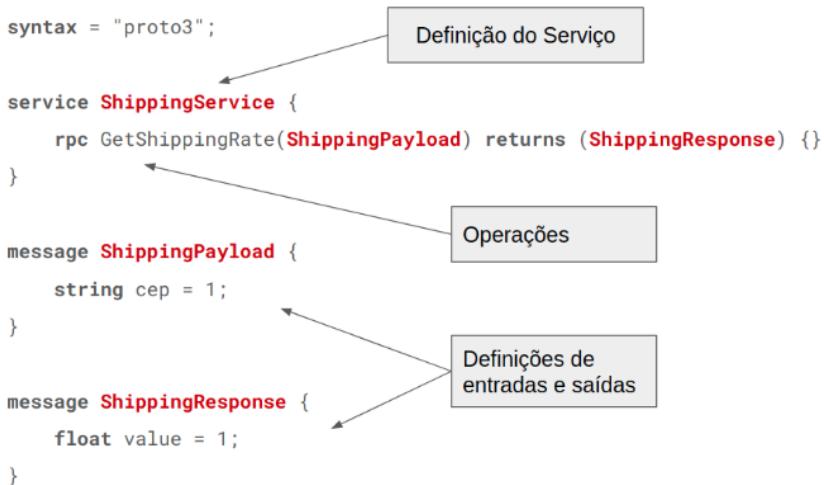
Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

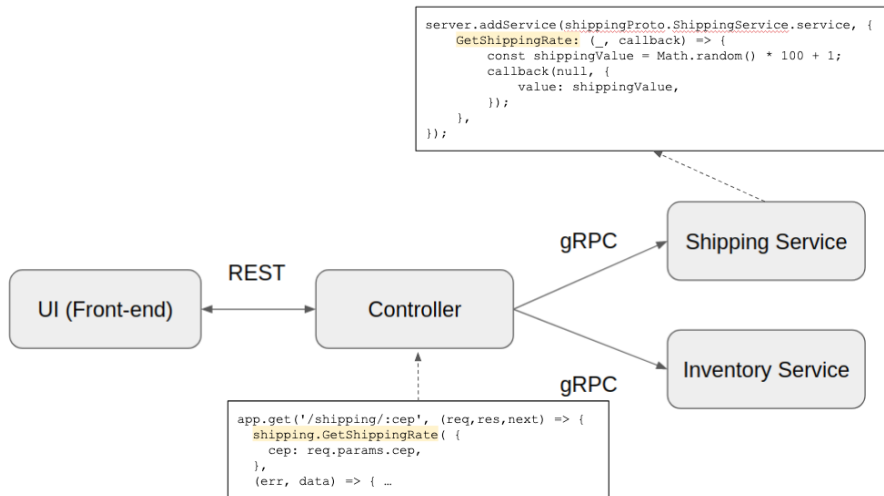
Exercícios

- ▶ Sistema que viabiliza comunicação entre aplicações
- ▶ Baseado em Chamada Remota de Procedimentos
- ▶ Inclui um protocolo binário para comunicação distribuída
- ▶ E também uma linguagem para definição de interfaces
 - Clientes: chamam métodos dessa interface
 - Servidores: implementam métodos

GRPC: LINGUAGEM PARA DEFINIÇÃO DE INTERFACES



EXEMPLO DE IMPLEMENTAÇÃO GRPC



Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

ARQUITETURA ORIENTADA A MENSAGENS

Arquitetura para aplicações distribuídas

Clientes não se comunicam diretamente com servidores

Mas com um intermediário: **fila de mensagens** (ou broker de mensagens)



Exemplos: IBM MQ, RabbitMQ, Apache ActiveMQ, HornetQ, ...

VANTAGEM #1: TOLERÂNCIA A FALHAS

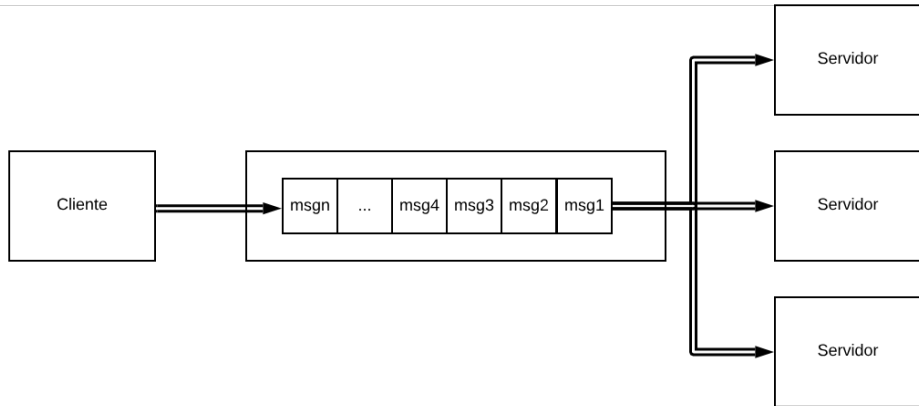
Não existe mais mensagem: “servidor fora do ar”

Assumindo que a fila de mensagens roda em um servidor bastante robusto e confiável



VANTAGEM #2: ESCALABILIDADE

Mais fácil acrescentar novos servidores (e mais difícil sobrecarregar um servidor com excesso de mensagens)



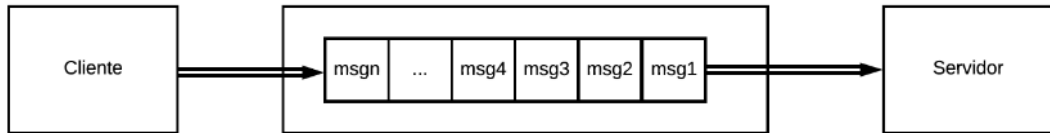
COMUNICAÇÃO ASSÍNCRONA

Comunicação entre clientes e servidores é **assíncrona**

Cria um acoplamento fraco entre clientes e servidores

Desacoplamento no espaço: clientes não conhecem servidores e vice-versa

Desacoplamento no tempo: clientes e servidores não precisam estar simultaneamente no ar



Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

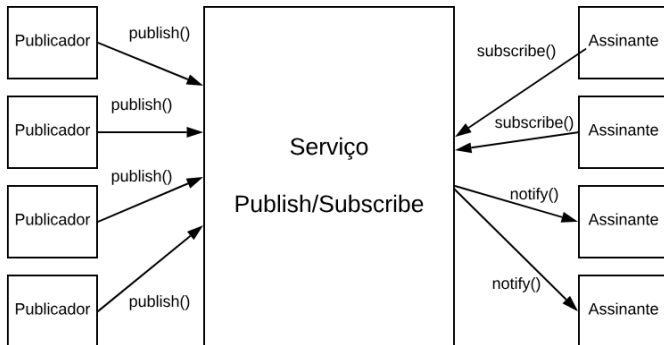
ARQUITETURA PUBLISH/SUBSCRIBE

“Aperfeiçoamento” de fila de mensagens

Mensagens são chamadas de **eventos**

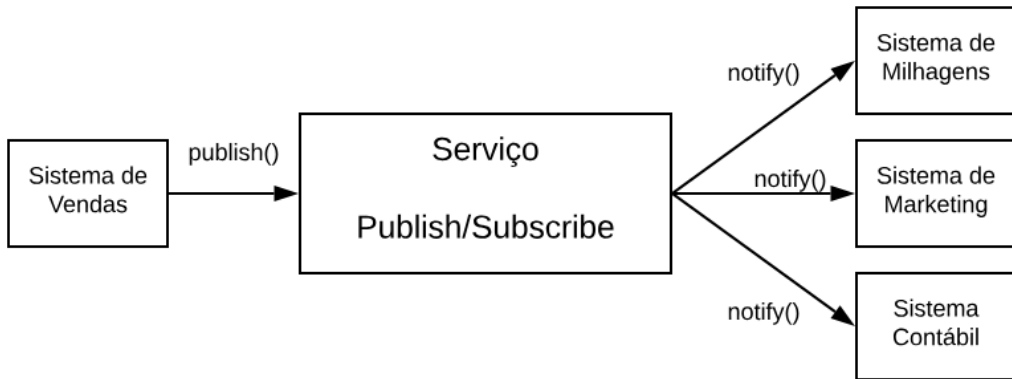
CONCEITO PUBLISH/SUBSCRIBE

Sistemas podem: assinar eventos; publicar eventos; serem notificados sobre a ocorrência de eventos



Exemplo: Apache Kafka

EXEMPLO: SISTEMA DE COMPANHIA AÉREA



Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

PIPES E FILTROS

Programas são chamados de **filtros** e se comunicam por meio de **pipes** (que agem como buffers)

Arquitetura bastante flexível. Usada por comandos Unix.

Exemplo:

```
$ ls | grep csv | sort
```

CLIENTE/SERVIDOR

- ▶ Muito comum em serviços básicos de redes
- ▶ Exemplos:
 - Serviço de impressão
 - Serviço de arquivos
 - Serviço Web



PEER-TO-PEER

Todo nodo pode ser cliente e/ou servidor
Isto é, pode ser consumidor e/ou provedor de recursos

Exemplo: sistemas para compartilhamento de arquivos (usando BitTorrent); Blockchain



Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

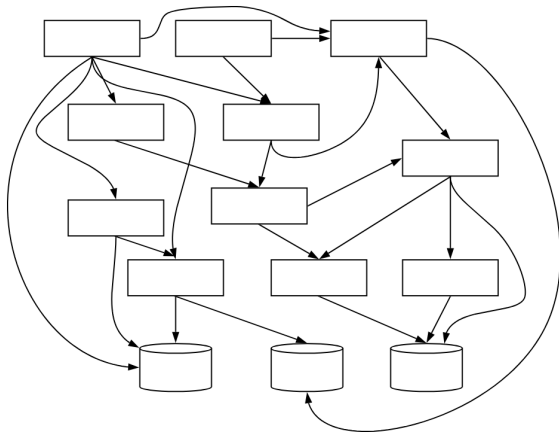
ANTI-PADRÃO

Modelo que não deve ser seguido

Revela um sistema com sérios problemas arquiteturais

BIG BALL OF MUD

Um módulo pode usar praticamente qualquer outro módulo do sistema; ou seja, sistema é uma “bagunça”



OU SE PREFERIR...



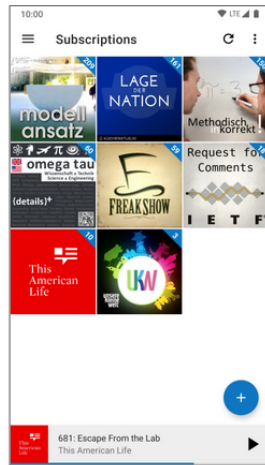
OU SE PREFERIR...

Uma **maçaroca**!

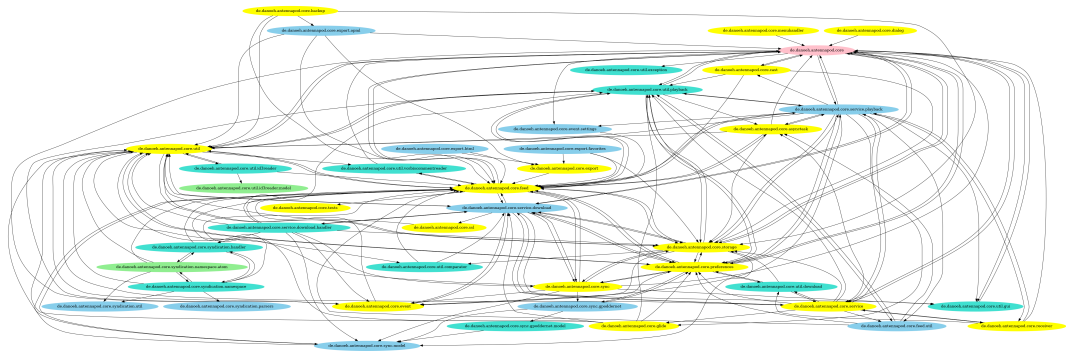


EXEMPLO DE REMODULARIZAÇÃO

AntennaPod: sistema open-source para tocar podcasts
Android e Java
<https://antennapod.org>



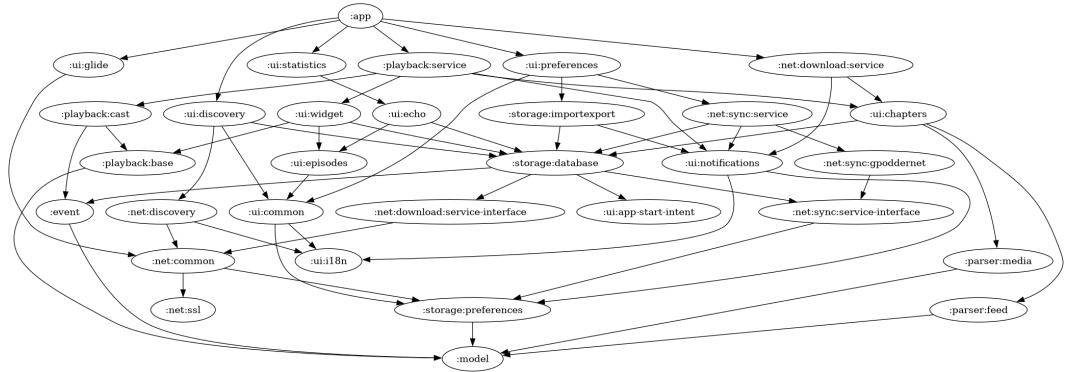
NOVEMBRO, 2020



<https://antennapod.org/blog/2024/05/modernizing-the-code-structure>

COMENTÁRIOS DOS DESENVOLVEDORES

“To test the database, for example, one normally wouldn’t have to launch the full app. However, because the database basically depended on everything else, most of our tests required starting up a full Android device.”



<https://antennapod.org/blog/2024/05/modernizing-the-code-structure>

Arquiteturas de Software

Introdução à Arquitetura

Arquitetura em Camadas

Arquitetura Model-View-Controller (MVC)

MVC Web

Single Page Applications (SPAs)

Monolitos

Microserviços

gRPC

Arquitetura Orientada a Mensagens

Arquitetura Publish/Subscribe

Outros Padrões Arquiteturais

Anti-padrões Arquiteturais

Exercícios

EXERCÍCIO 1

Qual a provável arquitetura dos seguintes sistemas? Justifique brevemente sua resposta.

- a) Microsoft Excel (versão para desktop)
- b) Nubank App (mobile)
- c) Twitter Web (front-end)
- d) Google Slides (front-end)
- e) Twitter (backend)
- f) Moodle

EXERCÍCIO 2

Responda sobre microsserviços:

- a) Por que uma arquitetura baseada em microsserviços oferece flexibilidade para os times colocarem seu código em produção de forma independente?
- b) Por que isso é menos viável em uma arquitetura monolítica? Ou seja, por que não é recomendável que um time, ao fazer uma modificação em um monolito, coloque-a em produção imediatamente?
- c) Por que microsserviços não devem compartilhar o mesmo BD?

EXERCÍCIO 3

Suponha uma empresa de streaming que deseja implementar um sistema detector de problemas de qualidade em seus vídeos (por exemplo, problemas nas legendas, no áudio, congelamento de imagens, etc.). Os vídeos estão todos armazenados em um sistema de storage, isto é, em memória secundária. A empresa está avaliando duas arquiteturas:

Arquitetura #1: cada detector é um microsserviço, que recebe o nome do vídeo como parâmetro, carrega ele do storage e executa um algoritmo de detecção de problemas de qualidade nesse vídeo.

Arquitetura #2: os detectores são módulos de um monolito. Então, o vídeo é carregado uma única vez do storage para a memória principal e compartilhado por todos os detectores de qualidade.

EXERCÍCIO 3 (CONTINUAÇÃO)

Supondo que a empresa de streaming tem milhões de clientes (ou seja, é uma empresa do porte da Amazon Prime Video) qual arquitetura você considera mais escalável? Justifique.

EXERCÍCIO 3 (CONTINUAÇÃO)

Este exercício é baseado em um artigo do blog da Amazon Prime Video.



EXERCÍCIO 3 (CONTINUAÇÃO)

Problema Original

- ▶ Alto custo: A arquitetura de microserviços distribuídos, orquestrada pelo AWS Step Functions, era cara devido às múltiplas transições de estado por segundo de stream.
- ▶ Armazenamento caro: O uso de um bucket S3 como armazenamento intermediário para frames de vídeo gerava um grande número de chamadas caras (Tier-1 calls).
- ▶ Limites de escala: A orquestração com AWS Step Functions rapidamente atingia os limites da conta, impedindo o monitoramento de milhares de streams.

EXERCÍCIO 3 (CONTINUAÇÃO)

Solução

- ▶ Arquitetura monolítica: A equipe consolidou todos os componentes em um único processo.
- ▶ Transferência de dados na memória: A nova arquitetura eliminou a necessidade de um bucket S3, permitindo que a transferência de dados ocorresse na memória, o que reduziu custos e otimizou o desempenho.
- ▶ Orquestração interna: A orquestração passou a ser controlada dentro de uma única instância.
- ▶ Escalabilidade vertical e horizontal: Para superar a limitação de escalabilidade vertical de uma única instância, o serviço foi clonado várias vezes, com cada cópia sendo parametrizada para um subconjunto diferente de detectores, e uma camada de orquestração leve foi implementada para distribuir a carga.

EXERCÍCIO 3 (CONTINUAÇÃO)

Lições Aprendidas

- ▶ Priorizar escalabilidade e custo: É crucial entender as necessidades de escalabilidade e o impacto nos custos antes de projetar um sistema distribuído.
- ▶ Revisitar a arquitetura: A reavaliação periódica da arquitetura é fundamental para otimizar desempenho, custo e escalabilidade.
- ▶ Transferência eficiente de dados: Otimizar a transferência de dados, preferencialmente usando a memória interna, é vital para reduzir custos e melhorar a performance.

Este documento está licenciado sob a Licença Atribuição-NãoComercial-Compartilhalgual 4.0 Internacional Creative Commons. Para visualizar uma cópia desta licença, visite <http://creativecommons.org/licenses/by-nc-sa/4.0/> ou mande uma carta para Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

